

DSC 40B: THEORETICAL FOUNDATIONS OF DATA SCIENCE II

HASHING III



DOUBLE HASHING

$i = 0, 1, 2, 3$

$p = 10^9$
 $m = 11$
 $x = 31$

$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$. The step size is itself a hash of the key, so two keys that start in the same slot walk away from it at different speeds.

$M = 11$, $H_1(K) = K \bmod 11$, $H_2(K) = 1 + (K \bmod 10)$

$20 \div 11$

KEY	H_1	H_2	PROBE SEQUENCE	LANDS	PROBES
20	9	1	9 free $9 + (0 \cdot 1)$ $9 + 0$	9	1
34	1	5	1 free $1 + 1(6)$	1	1
45	1	6	1, $1+6 \rightarrow 7$ free	7	2
31	9	2	9, $9+2 \rightarrow 0$ free	0	2
42	9	3	9, $9+3 \rightarrow 1$, $9+6 \rightarrow 4$ free $9 + 2(3)$ $15 \div 11$	4	3

31	34	45	1	42	20	
0	1	2	3	4	9	10

$H_2(K)$ MUST NEVER BE 0

A step size of zero probes the same slot forever. Ensure h_2 returns a value in $1 \dots m-1$.

$H_2(K)$ MUST BE COPRIME TO M

Otherwise the sequence visits only $m / \gcd(h_2(k), m)$ slots, by the same gcd argument as the division method. Taking m prime makes every step size valid.

31 and 42 both hash to slot 9, but they leave it in different directions. Under linear and quadratic probing they would have followed the same path. That difference is what removes secondary clustering.

$O(M^2)$ DISTINCT PROBE SEQUENCES

Linear and quadratic offer only m , one per starting slot. This comes closest to the uniform hashing ideal!



SAME KEYS, THREE SCHEMES

$m = 11$, $h(k) = k \bmod 11$, inserting 20, 34, 45, 31, 42 in that order. Identical input, three different final tables. A dot marks a slot that stayed empty.

	SLOT 0	SLOT 1	SLOT 2	SLOT 4	SLOT 7	SLOT 9	SLOT 10	PROBES
Linear probing	42	34	45	.	.	20	31	1,1,2,2,3 = 9
Quadratic probing	.	34	45	.	42	20	31	1,1,2,2,4 = 10
Double hashing	31	34	.	42	45	20	.	1,1,2,2,3 = 9

Handwritten notes:

- Red arrows: 3 (from 42 to Slot 0), 5 (from 45 to Slot 7), 6 (from 20 to Slot 9).
- Blue circle around 20 in Slot 9.
- Red circles around 42 in Slot 4 and 45 in Slot 7.
- Red lines under Slot 0-2 and Slot 9-10 in Linear probing.
- Red line under Slot 0-2 in Double hashing.
- Blue text: $h_2(k)$ with an arrow pointing to Slot 0 in Double hashing.
- Green box around Slot 10 with blue text: $i = 0, 1, 2$ and $i = 0, 1, 4, 9$.

WHAT THE EXAMPLE DOES SHOW

Linear probing produces one contiguous run at slots 0-2 and another at 9-10. The other two schemes scatter their keys, which is exactly the clustering difference described.

WHAT IT DOES NOT SHOW

At $n = 5$ and $\alpha \approx 0.45$, the three schemes cost almost the same. The differences between them are asymptotic and only appear as α approaches 1.

Do not conclude from this table that linear probing is fine. It ties here and it is catastrophically worse at $\alpha = 0.9$. The next two slides are where that shows up.



EXPECTED PROBES UNDER OPEN ADDRESSING

The bound is no longer linear in α . It is a pole. The cost does not grow steadily as the table fills; it blows up.

THE ASSUMPTION

Uniform hashing: each key's probe sequence is equally likely to be any of the $m!$ permutations of the slots. This is strictly stronger than SUHA, no real scheme achieves it exactly, and double hashing comes closest.

```
# Unsuccessful search. Each probe hits an occupied
# slot with probability about alpha, so the probe
# count is geometric with success rate 1 - alpha:
```

$$\begin{aligned} E[\text{probes}] &= \sum_{i \geq 0} \alpha^i \\ &= \underline{1 / (1 - \alpha)} \end{aligned}$$

```
# Successful search, averaged over insert order:
```

$$E[\text{probes}] = (1/\alpha) * \ln(1 / (1 - \alpha))$$

LINEAR PROBING IS WORSE THAN THE IDEAL

Primary clustering costs real time. Knuth's bounds are $\frac{1}{2}(1 + 1/(1-\alpha)^2)$ for an unsuccessful search and $\frac{1}{2}(1 + 1/(1-\alpha))$ for a successful one.

Chain: $(1 + \alpha^2) \rightarrow 1 - (1 - \alpha)$ $\alpha = 1$

READ THE SQUARED TERM

That square is the signature of clustering. At $\alpha = 0.9$ the uniform-hashing ideal costs 10 probes and linear probing costs about 50, five times worse at the same load factor.



THE COST OF A FULL TABLE

Expected probes for an unsuccessful search. This table is the reason every real implementation resizes long before the table fills.

α	CHAINING $1 + \alpha$	UNIFORM $1/(1-\alpha)$	LINEAR $\frac{1}{2}(1 + 1/(1-\alpha)^2)$
0.50	1.5	2.0	2.5
0.75	1.8	4.0	8.5
0.90	1.9	10.0	50.5
0.95	2.0	20.0	200.5
0.99	2.0	100	5000.5

Handwritten annotations: A red arrow points down the left side of the table. Blue underlines are under 1.5, 2.0, 2.5, 1.8, 4.0, 8.5, 1.9, 10.0, 50.5, 200.5, and 5000.5. A red dashed box encloses the last three rows. A red circle highlights the value 5000.5.

THE PRACTICAL RULE

Resize before α gets large. Java's HashMap resizes at 0.75; Python's dict resizes at roughly 0.66.

WHY CHAINING LOOKS FLAT

Because it is linear in α and α is capped at 1 in this table. Chaining is the only scheme here that can legally exceed $\alpha = 1$ at all.

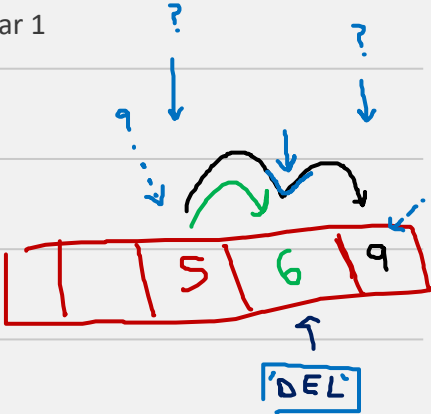
Read the last row. At 99% full, one lookup costs five thousand probes under linear probing. The table went from comfortable to unusable between $\alpha = 0.9$ and $\alpha = 0.99$, a change of nine per cent in occupancy, and a hundredfold change in cost.



CHAINING VERSUS OPEN ADDRESSING

Both are expected constant time and both have a linear worst case. Everything that separates them is on this table.

	SEPARATE CHAINING	OPEN ADDRESSING
Where keys live	In linked lists outside the table	In the table array itself
Load factor	Any $\alpha > 0$, and it degrades gracefully	Requires $\alpha < 1$, and degrades sharply near 1
Expected search	$\Theta(1 + \alpha)$	$\Theta(1 / (1 - \alpha))$
Worst case	$\Theta(n)$	$\Theta(n)$
<u>Deletion</u>	<u>A plain unlink</u>	<u>Needs tombstones, which accumulate</u>
Memory	One extra pointer per entry; slots may sit empty	No pointers; contiguous and compact
Cache behaviour	Poor: pointer chasing through memory	Good: sequential access within a cluster
Used by	Java HashMap, most textbook treatments	Python dict, Go map, most modern libraries



Rule of thumb: deletion-heavy or unpredictable load, use chaining. Memory-constrained and read-heavy with a load factor you control, use open addressing. Neither row fixes the worst case, which is what the last section is about.



BEYOND EXPECTED CONSTANT TIME



PERFECT HASHING

If the key set is fixed and known in advance, we can eliminate collisions completely and get worst-case constant-time lookup.

01 Outer table of size $m = n$

Hash all n keys with a universal hash function. Slot j receives n_j keys, and collisions still happen at this level.

02 Inner table of size n_j^2 for slot j

Give each slot its own universal hash into a table of size n_j^2 . Quadratic size makes a collision-free choice likely; redraw until you find one.

03 Lookup is exactly two hashes

Compute the outer slot, then the inner slot. No probing, no scanning, no chains. Worst-case $\Theta(1)$, unconditionally.

WHY n_j^2 IS THE RIGHT SIZE

With a table of size n_j^2 and a universal family, the expected number of colliding pairs is at most $C(n_j, 2)/n_j^2 < 1/2$. By Markov, a random draw is collision-free with probability above $1/2$, so a couple of tries suffice.

SPACE IS STILL LINEAR

Squaring each n_j sounds ruinous, but $E[\sum_j n_j^2] < 2n$ under a universal outer hash. Total space is $\Theta(n)$.

THE RESTRICTION

One new key can invalidate the whole construction. This is a read-only data structure.



CUCKOO HASHING

Worst-case constant-time lookup for a table you can still insert into. The name comes from the bird that pushes other eggs out of the nest.

01 Every key has exactly two possible homes

Two tables and two hash functions. Key k lives at $T_1[h_1(k)]$ or at $T_2[h_2(k)]$ and nowhere else, so lookup and delete check two slots and stop, giving worst-case $\Theta(1)$.

```
lookup(k):           # exactly 2 probes
    if T1[h1(k)] == k: return T1[h1(k)]
    if T2[h2(k)] == k: return T2[h2(k)]
    return None
```

02 Insert by eviction

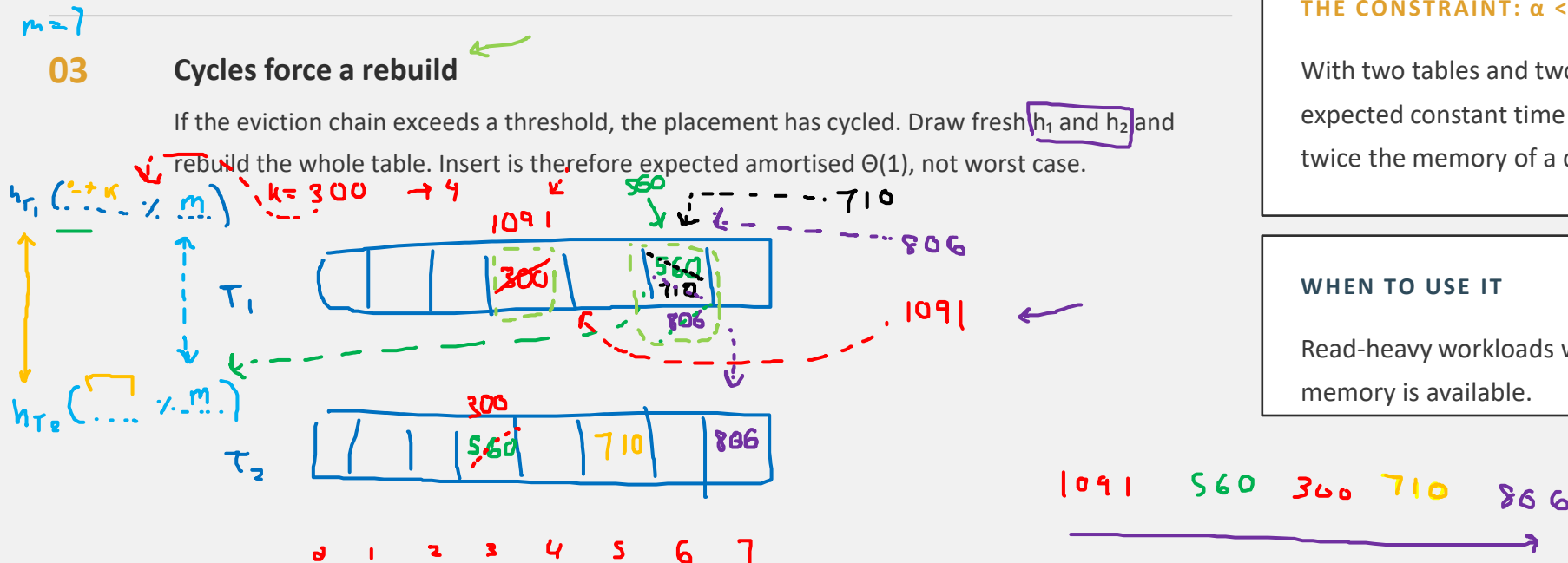
Place k at $T_1[h_1(k)]$. If that slot is occupied, evict its occupant and re-place it in its alternate table, which may evict another key, and so on down the chain.

THE CONSTRAINT: $\alpha < 1/2$

With two tables and two hash functions, insertion succeeds in expected constant time only below half capacity. More than twice the memory of a comparably loaded table.

03 Cycles force a rebuild

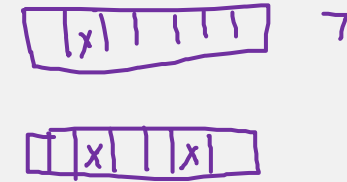
If the eviction chain exceeds a threshold, the placement has cycled. Draw fresh h_1 and h_2 and rebuild the whole table. Insert is therefore expected amortised $\Theta(1)$, not worst case.



WHEN TO USE IT

Read-heavy workloads where a slow lookup is unacceptable and memory is available.

$n = 3$



CHOOSING A STRATEGY

Everything from both lectures in one table. The top four are expected constant time; the bottom two are worst-case constant time, and each gives something up for it.

STRATEGY	LOOKUP (EXP.)	LOOKUP (WORST)	MAX α	USE WHEN
Separate chaining	$\Theta(1 + \alpha)$	$\Theta(n)$	any	General purpose; frequent deletions; load factor hard to predict
Linear probing	$\Theta(1/(1-\alpha))$	$\Theta(n)$	< 1	Cache-sensitive workloads with α reliably kept below about 0.7
Quadratic probing	$\Theta(1/(1-\alpha))$	$\Theta(n)$	$< 1/2^*$	Primary clustering is the bottleneck and m can be chosen prime
Double hashing	$\Theta(1/(1-\alpha))$	$\Theta(n)$	< 1	The highest load factors among the probing schemes
Perfect hashing	$\Theta(1)$	$\Theta(1)$	n/a	The key set is static and known before the table is built
Cuckoo hashing	$\Theta(1)$	$\Theta(1)$	$< 1/2$	Lookups must never be slow and the memory is available

* The $\alpha < 1/2$ bound with m prime is what guarantees quadratic probing terminates. It can be run above that threshold, without the guarantee.

THE ONE-LINE VERSION

Worst-case constant time is achievable, but always by surrendering something. Perfect hashing gives up insertion; cuckoo hashing gives up half the table. There is no free version.



CLOSING THOUGHTS

